Lab report

Student Name: Xirui Zhu
Student ID: 2530655

Component A: Staff Interview

Jason Evans:Interview Script

Jason Evans is the academic advisor responsible for reviewing credit transfer applications. Students must submit an application if they wish to transfer credits from off-campus courses into required courses in the TECHIN program. Jason's role is to assess whether the content of the off-campus courses is identical to the content of the required courses in the TECHIN program.

Interview Notes

Q: Can you walk me through what happens when a student submits a course petition?

Students upload their application materials themselves, typically one set per application form. They need to submit their off-campus course schedules so I can compare them with TECHIN's schedules.

Note: The materials uploaded are entirely up to the students, format and completeness may vary.

Q: What are you actually looking for when you compare the two syllabi?

I will check if they are substantially equivalent; I consider a similarity of around 90% acceptable. I will consider four main aspects: learning objectives, the institution of instruction, the topics covered, and whether the courses use similar projects or exams as assessment criteria.

Note: The equivalence criteria must be clearly understood.

Q: What happens when one course covers multiple TECHIN equivalencies?

If a student points out that an off-campus course covers multiple Technin courses, I will compile all equivalent courses into a single file for verification. Each course is allowed a maximum of three equivalent courses.

Note: The files are merged manually, there is no dedicated tool for this task.

Q: How do you communicate with students during this process?

I send individual emails to each student instead of mass emails because each student's situation is different. Also, since not all students speak English, communication can

sometimes be difficult. I use different systems, and the responses I receive vary, making it difficult to track the status of each application.

Note: Direct email communication overcomes the language barrier, but there is no centralized tracking system.

Q: Is there anything that slows you down the most?

Frankly, the problem lies in reliability. Emails aren't always reliable; after I send an email, I don't know if the student has received it. Furthermore, I have to manually check each student's total credits to ensure they don't exceed nine credits. This is a fixed limit, but the lack of an automatic check forces me to verify it manually every time.

Note: There is no automatic credit limit check; manual verification is required.

Q: What does the approval process look like after your review?

After I complete my review, I send my recommendation to the instructor for approval. Then it goes to equivalence faculty review. Once everything is approved, the course gets highlighted in the student's transcript, and I send a result email to the student.

Emotional Journey Map

🔴**Frustration Peak** 🟡**Uncertainty Zone** 🟢**Delight Moment**

Step	Action	Emotion
1	Student uploads documents	🟡 format and completeness are unpredictable
2	Jason opens and checks the submitted syllabus	🟢clear starting point when documents are complete
3	Compare student syllabus and TECHIN syllabus on 4 keywords	🟢 90% match rule gives clear decision criteria
4	If multiple courses, it has to merge into one combined file	🔴manual merging with no dedicated tool, tedious and error-prone
5	Check student's email domain	🟡 inconsistent; non-.edu accounts sometimes unreachable

6	**Email student individually (not in group)**	**🔴time consuming, repeated work, and some students reply in other languages**
7	**Track petition status across multiple systems**	**🔴different systems give different responses, no single source of truth**
8	**Manually verify 9 credit cap for each student**	**🔴no automation and must cross reference manually every time**
9	**Send recommendation to instructor for approval**	**🟡outcome depends on instructor availability and response time**
10	**Equivalence faculty review**	**🟡 timeline and criteria can vary by faculty**
11	**Highlight course in transcript upon approval**	**🟢 clear, satisfying closure when everything is in order**
12	**Send result email to student**	**🟢 closing the loop, student gets their answer**

Problem Statement

“When Jason needs to check whether the syllabus of a course application is equivalent, he currently has to manually compile documents, send emails to each student individually through multiple unreliable systems, and manually check credit limits, which leads to repetitive administrative work, inconsistent follow up, and delays in student results.”

If then flowchat

Design Implecation

Reduce frustration: Automatically monitor credit limits, such as alerting when approaching the 9-credit limit; provide structured document upload templates to ensure consistent submission formats, centralize application tracking on a single dashboard to eliminate multiple email exchanges; and offer translation assistance for convenient multilingual communication.

Maintain a positive experience: Credit equivalence decisions must be made personally Jason's four keyword analysis framework is based on his expertise, not off theshelf formulas. Maintain personalized outcome communication while streamlining the process.

Eliminate ambiguity: Standardize the merging process for multiple courses using tools that automatically apply the 3-credit rule, and provide progress updates to ensure Jason and students are always informed about their application status.

Component B: Lab

Recall Prompt

Before writing my first proposal today, I carefully considered the problem statement for Component A:
"When Jason needs to review course projects to verify their equivalence, he currently has to manually merge documents, send emails to students one by one through multiple unreliable systems, and manually check credit caps. This results in duplicated administrative work, inconsistent tracking, and delays in student grades."

Using the same format, I would like to describe the functionality I want the AI to create:
"When developers need to create a course application review tool, they currently spend a significant amount of time manually designing the user interface, data management, and workflow logic, which slows down prototyping and leads to inconsistent code structure. I want the AI to generate a Streamlit application that can handle document uploads, keyword comparison, and progress tracking."

Model Comparison

Test Prompt Used

Write a Python function that takes as input a list of student records (dictionaries containing the keys "name", "program", and "average") and returns only the students from a given program whose overall average is above a given threshold. Handle errors.

Model A: Claude (in Cursor)

```
def filter_students(
    students: list[dict],
    program: str,
    gpa_threshold: float
) -> list[dict]:
    """
    Filter students by program and minimum GPA.

    Args:
        students: List of student record dicts with 'name', 'program', 'gpa'
        program: Program name to filter by
        gpa_threshold: Minimum GPA (inclusive)

    Returns:
        List of matching student records

    Raises:
        TypeError: If students is not a list
        ValueError: If gpa_threshold is not a valid number
    """
```

```
    if not isinstance(students, list):
        raise TypeError(f"Expected a list, got {type(students).__name__}")
    if not isinstance(gpa_threshold, (int, float)):
        raise ValueError(f"gpa_threshold must be a number, got
{type(gpa_threshold).__name__}")

    result = []
    for i, student in enumerate(students):
        if not isinstance(student, dict):
            raise TypeError(f"Record at index {i} is not a dict: {student}")
        missing = [key for key in ("name", "program", "gpa") if key not in
student]
        if missing:
            raise KeyError(f"Record at index {i} is missing keys: {missing}")
        if student["program"] == program and student["gpa"] >= gpa_threshold:
            result.append(student)
    return result
```

Claude's explanation: clarifies the type of each parameter, describes the logic of the loop step by step ,explicitly lists which exceptions will be raised and when, and mentions that the comparison is inclusive (>=).

Model B: ChatGPT (GPT-4o)

```
def filter_students(students, program, gpa_threshold):
    """
    Returns students in the specified program with GPA above the threshold.
    """
    if not isinstance(students, list):
        raise TypeError("students must be a list")

    filtered = []
    for student in students:
        try:
            if student.get("program") == program and student.get("gpa", 0) >=
gpa_threshold:
                filtered.append(student)
        except Exception as e:
            print(f"Skipping invalid record: {e}")

    return filtered
```

ChatGPT's explanation provides a brief summary, mentioning that `.get()` is used to avoid KeyError, but it doesn't explain each line of code in detail. It's recommended to add unit tests later.

Observations

Code Style: Claude uses type annotations (list[dict], float) and detailed docstrings containing parameters, return values, and exception handling, bringing it closer to production standards. ChatGPT generates more concise and readable code, but without annotations. While easier for beginners to grasp, its documentation is inadequate.
Error Handling: Claude throws explicit and descriptive exceptions. ChatGPT uses wide try/except blocks within loops, silently ignoring erroneous records by displaying them. This

prevents crashes but masks data quality issues, which can be dangerous in real-world applications.
Quality of Explanations: Claude's explanations are more detailed, explaining each extreme case and the reasons behind the decisions made. ChatGPT's are more general and conversational, facilitating quick understanding of the overall logic, but less useful for debugging or code review.

Claude code setup

Step 1.1 Verify Week 1 Tools

```
$ git --version
git version 2.46.0

$ python3 --version
Python 3.12.3

$ streamlit --version
Streamlit, version 1.35.0

$ node --version
v20.14.0
```

All tools have been confirmed to be working correctly. The Node.js version is v20 (LTS), which meets the requirements.

Step 1.4 Launch Claude Code in Project

Project contains: app.py (the main Streamlit application – a tip calculator with a bill input field, a tip percentage slider, and tip distribution per person), requirements.txt (list of Streamlit and pandas dependencies), and .venv/ (virtual environment directory). The application currently runs in single-page mode and does not store any data.

Prompt1: Blurry
Prompt: Add a graph to my application

Output: The cursor generates a Matplotlib bar chart with hard-coded example data [10, 20, 30], imports matplotlib.pyplot, and uses st.pyplot(), but there are no titles or axis labels.

Actual Action: The wrong library was chosen (Matplotlib instead of Plotly), which generated erroneous data unrelated to the application. No interactive functionality was added. The application ran, but the results were completely wrong.

Prompt 2: Details

Prompt: Use Plotly to add a bar chart below the receipt summary. Display three bars:

"Bill Amount," "Tip Amount," and "Per Person." Extract the values from the following functions:

bill_amount, tip_amount, and per_person. Add a title, "Payment Details."

Add labels to both axes and enable tooltips to display the precise amounts.

Result: A Plotly px.bar() chart with the correct variables, labels, and tooltips, but hidden by default in the if st.button("Show Chart") function.

Output: Connected to real data, using the appropriate library. Incorrect assumption: The chart is hidden by the button.

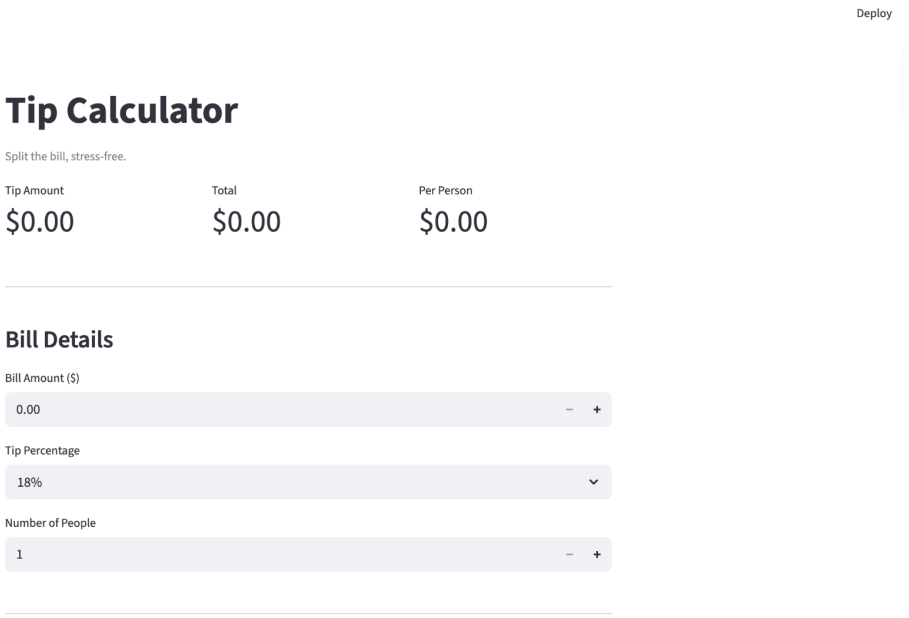
Prompt 3 Restricted

Prompt:

Add a Plotly bar chart directly below the receipt summary (instead of Matplotlib)
Always visible, not inside a button. Use variables bill_amount, tip_amount, and per_person. Palette: ["#1f77b4", "#ff7f0e", "#2ca02c"]. Title: "Payment Details". X-axis: "Category". Y-axis: "Amount (USD)". Hover tooltip:Formatted as "$X.XX". If bill_amount is 0, display st.info("Please enter the bill amountto view the chart."). Follows .cursorrules.

Output: Exact match ,Always visible, correct palette, $X.XX format tooltip, st.info() displayed when amount is 0, type tooltip and docstring from .cursorrules.Eliminates all incorrect assumptions. Behavioral restrictions (not inside the button) have the greatest impact.

App Screenshot



Component C: System Architecture

C.1 Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                          USER (Browser)                              │
│                     Interacts via Streamlit UI                       │
└──────────────────────────────┬───────────────────────────────────────┘
                               │ HTTP (localhost:8501)
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│                       app.py  (YOUR CODE)                            │
│  - st.set_page_config()                                              │
│  - UI layout: inputs, metrics, receipt, chart                        │
│  - st.session_state for persistent values                            │
│  - calls utils.py functions for computation                          │
└──────────────┬──────────────────┬────────────────────────────────────┘
               │                  │
               ▼                  ▼
┌──────────────────┐   ┌──────────────────────────────────────────┐
│   utils.py       │   │          External Libraries              │
│  (YOUR CODE)     │   │  Streamlit  │  Plotly  │  Pandas         │
│  - compute_tip()│   │  (UI fwk)   │  (charts)│  (data tables)  │
│  - compute_stats│   │  You call   │  You call│  You call them  │
│  - format_receipt│  │  them; you  │  them;   │  but don't own  │
└──────────────────┘   │  don't own  │  don't   │  them           │
                       │  Streamlit  │  own it  │                 │
                       └──────────────────────────────────────────┘
               │
               ▼
┌──────────────────────────────────────────────────────────────────────┐
│                   AI Tools (Boundary Layer)                          │
│                                                                      │
│  ┌──────────────────────┐    ┌──────────────────────────────┐       │
│  │  Cursor (IDE)        │    │  Claude Code (Terminal)      │       │
│  │  Reads: .cursorrules │    │  Reads: CLAUDE.md            │       │
│  │  Modes:              │    │  Commands: read/write files  │       │
│  │   Composer (Cmd+I)   │    │  Scope: whole project        │       │
│  │   Chat (Cmd+L)       │    │                              │       │
│  │   Inline (Cmd+K)     │    │                              │       │
│  └──────────────────────┘    └──────────────────────────────┘       │
│                                                                      │
│  Boundary: AI tools PROPOSE changes. YOU accept or reject.           │
│  They cannot push to GitHub or run your app on their own.            │
└──────────────────────────────┬───────────────────────────────────────┘
                               │
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│                          GitHub                                      │
│  - Remote code storage                                               │
│  - Version history (git log, git blame)                              │
│  - You push; GitHub handles storage, access, versioning              │
│  - Boundary: you control what gets pushed via git add/commit         │
└──────────────────────────────────────────────────────────────────────┘
```

Component Ownership Summary:

Component	Who owns it	What the boundary looks like
app.py	me	I write it; AI proposes, I accept
utils.py	me	Pure Python logic; no UI dependency
Streamlit	Third party	st.* functions,Streamlit handles rendering
Plotly	Third party	Ibuild fig objects; Plotly handles SVG/JS output
.cursorrules / CLAUDE.md	me	instructions to AI tools; they read but cannot modify
Cursor / Claude Code	Third party ,AI tools	They read my files and propose changes
GitHub	me	Stores my git objects, me control what is pushed

C.2 Design Decision

Field	Entry
Decision	Streamlit coding standards and conventions are stored in the .cursorrules file, while the project description, technology stack, and execution commands are stored in the CLAUDE.md file. Only prompts for specific instructions used for particular tasks are retained.
Alternatives considered	(1) Put all the contents in the .cursorrules file. (2) Put all the contents in each prompt. (3) Do not use any files and rely on the context of the prompt each time.
Why I chose this	The `.cursorrules` file concerns cursors and focuses on code, the `CLAUDE.md` file concerns Claude's code and focuses on the project context. Using them simultaneously can mislead users about which tool is being used to read what.

	The prompt messages are temporary: they disappear after each session, making it impossible to maintain consistency between sessions.
Trade off	Maintaining two separate context files means they can become out of sync. If I update the technology stack in CLAUDE.md but forget to update the .cursorrules file, the context of one of the tools will become outdated. A single-file structure is easier to maintain but sacrifices tool-specific optimizations.
When I would choose differently	For one off projects or short-term prototypes, I would omit these two files and directly use configuration hints. Maintaining configuration files is too much work and not worth the effort. For team projects, I would add a third file (AI_CONTEXT.md) for both tools to read, thus achieving standardization and centralization.

Conflict Resolution: What Wins When Files Contradict?

During testing, I found that when the `.cursorrules` file specifies "Use Plotly" while the tooltip is "Use Matplotlib", the cursor follows the tooltip instead of the rule file. This makes sense: the tooltip is an explicit, direct instruction, while the rules are the default settings. The hierarchy is:

Explicit tooltip instructions > `.cursorrules` file > Model's default behavior

Implication for design: Put non-negotiables in config files . Put task-specific choices in the prompt. Never contradict yourself between the two resolve conflicts in the config file first.

Information belongs in One File

Information	Right place	Wrong place	Why
Coding style	.cursorrules	Prompt	Applies to every generated function , this should be automatic, not restated each time
"Run with streamlit run app.py"	CLAUDE.md	.cursorrules	This is operational context for Claude Code, not a coding convention
Target audience	CLAUDE.md	.cursorrules	Project context, not code style

Color palette	.cursorrules	CLAUDE.md	Code level styling standard, Cursor needs it and Claude Code rarely generates charts
API keys and secrets	Neither	Both	Never in any AI readable context file

Component D:Validation Experience

Results

#	Prompt Used	Without Config , before	With Config, After	What Changed	Better / Worse / Same
1	"Add a helper function to utils.py that takes a list of numbers and returns mean, median, and std deviation as a dictionary."	Generated function with no type hints, no docstring, used numpy (not in requirements .txt), single-letter variable n for the list	Generated function with list[float] type hint on input and dict[str, float] on output, full Google-style docstring with Args/Returns/Raises, used statistics module (stdlib), descriptive variable name values	Added type hints, docstring, switched from numpy to stdlib, renamed variable — all per .cursorrules	Better
2	"Explain the st.session_state pattern used in this app."	Generic explanation: "st.session_state is a dictionary-like object that persists values between reruns. It is useful for maintaining state in Streamlit apps." No	Explanation referenced the specific variables in the app (bill_amount, tip_rate, num_people) and connected them to the tip calculator workflow: "In your Tip Calculator, session_state prevents the receipt summary from resetting when the user adjusts the number of people."	Used project context from CLAUDE.md to ground the explanation in the actual app	Better

		project context.			
3	"Add error handling so the app doesn't crash if the user enters a negative bill amount."	Added a bare if bill_amount < 0: st.write("Invalid input") with no styling, no early return, placed after the calculation (too late to prevent the error)	Added if bill_amount < 0: st.error("Bill amount cannot be negative. Please enter a positive value."); st.stop() placed before the calculation, using st.error() per Streamlit conventions in .cursorrules and st.stop() to halt the rerun cleanly	Used st.error() instead of st.write(), added st.stop(), moved validation before computation — all per .cursorrules Streamlit conventions	Better

Smoke Test Results

#	Runs without error	Core path works	Invalid input handled
1 (compute_stats)	yes,streamlit run app.py passed	compute_stats([10, 20, 30]) returned {'mean': 20.0, 'median': 20.0, 'std_dev': 8.16}	compute_stats([]) raises ValueError: Cannot compute statistics on an empty list.
2 (explanation)	no, explanation only	no	no
3 (error handling)	yes,no crash on negative input	positive bill works normally	5 shows red st.error() and halts; 0 shows info message from chart section

Quality Gate Checklist

[x] Smoke test completed: Each generated function runs, core behavior works once, and invalid input is handled without crashing

[x] 3 prompts tested: Recording table has 3 complete rows with before/after observations

[x] Differences are specific: Each "What Changed" entry describes a concrete observable change

[x] Config files restored: .cursorrules and CLAUDE.md are back in place; re-ran Prompt 1 to verify

[x] Honest assessment: All 3 prompts showed improvement. No "Same" results, but I note that Prompt 2 is harder to measure objectively, the improvement is real but relies on judgment, not a passing or failing test

[x] Prompts are documented: Exact prompt text recorded in the table above; reproducible by anyone in the same project directory

Testing Concept: Regression Testing

Configuration files are inputs to the AI, not just documentation. Changing .cursorrules or CLAUDE.md is functionally equivalent to changing a parameter in a function ,it changes outputs. This means:Config files should be version controlled so changes are tracked. If AI output quality suddenly drops on a prompt that used to work, check whether a config file changed. In a team setting, config file changes should be reviewed like code changes, they affect every AI-generated output going forward. The most surprising finding: the biggest quality gain was not from style rules but from project-grounded explanations. When CLAUDE.md gave Claude Code the app's purpose and user context, its explanations became actionable rather than generic. This suggests that project context is as valuable in config files as coding standards.

Component E:Applied Chanllange

1. Five Sentence Guidelines

Build a single-page Streamlit application called “GIX Career Activity Eligibility Checker”. This application requires users to input three pieces of information: a dropdown menu to select a program (options: MSTI, MDE, MBA, EMBA, Other), a text input box to select the graduation quarter (format: “Winter 2025”, “Spring 2025”, “Summer 2025”, “Fall 2025”), and a radio button to select CPT status (Valid, Invalid, Not Applicable). Based on this input, the application displays a five-line results table, each row representing an activity (mock interview, resume review, employer focus group, social event, CPT information session), with each row displaying “Eligible” or “Ineligible” and a sentence explaining the reason. Eligibility rules are as follows: Mock interviews and resume reviews are open to all accredited programs; employer focus groups require graduation within the last four quarters; social events are open to everyone, CPT information sessions require valid CPT eligibility. If the course type is "Other", the graduation quarter cannot be parsed, or any required information is empty, the application will display a prominent warning to guide the user to contact an advisor rather than auto deciding their eligibility.

2. Two Implementations

Implementation A , Cursor eligibility_cursor.py
Implementation B, ChatGPT eligibility_chatgpt.py

3. Project- AI Configuration File

.cursorrules in project root. Key rules:
Python 3.11+, type hints required on all functions
Google-style docstrings, Plotly only (never Matplotlib), color palette ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]
st.error() + st.stop() for unrecoverable errors, No hardcoded data, no numpy

4. Comparison Table

Dimension	Cursor (with .cursorrules)	ChatGPT (no config)
Code structure	parse_graduation_quarter() + compute_eligibility() + UI section	Single st.button() block, all inline
Type hints	Yes — all functions annotated	No
Docstrings	Yes — Google style	No
Edge case: "Other" program	st.warning() + st.stop() — no table rendered	st.warning() — no table
Edge case: bad quarter format	Returns None → warning + st.stop()	st.error() with format hint
Edge case: blank quarter	st.info() + st.stop() before any parsing	st.warning() after button click
Display	st.dataframe() with green/red Styler highlighting	st.table() — static, no color
Runs on first try	✅ Yes	✅ Yes

5. Decision Flowchart

```
Student Enters: Program / Graduation Quarter / CPT Status]
                v
      [Is program recognized? (MSTI, MDE, MBA, EMBA)]
          |                    |
         YES                  NO
          |                    |
          v                    v
 [Is graduation quarter        🟡 "Program not found.
  within next 4 quarters?]        Contact advisor."
    |          |
   YES         NO
    |          |
    v          v
 [Eligible for  🟡 "Check back when
 time-sensitive   within 4 quarters
 events]          of graduation"]
    |
    v
 [Is CPT status = Active?]
    |          |
   YES         NO
    |          |
    v          v
 [Eligible for  [Not eligible for
 Employer Panels  CPT-specific
 + CPT Sessions]  events]
    |
    v
 [Show results table: 5 events × Eligible / Not Eligible / Reason]
    |
    v
```

*** 🔴 HUMAN REVIEW — system must NOT auto-decide: ***
· Student is on leave of absence
· Graduation quarter is disputed or under petition
· CPT status is "Pending" (not Active or Inactive)
· Dual-degree program not in recognized list
→ Display: "Your situation requires advisor review.
Please contact GIX Career Services directly."

Color key: 🟢 Green = clear auto-decision · 🔴 Red = human review required · 🟡 Yellow = flag + redirect

6. Smoke Test Evidence

Implementation A ,Cursor (eligibility_cursor.py):

Runs: streamlit run eligibility_cursor.py ,no import errors, app loads
Core path works: MSTI + "Spr 2026" + Active , 5 rows shown with correct eligibility
Invalid input handled: "Other" , st.warning() + st.stop(), no table, no crash; "xyz" as quarter , st.warning() + st.stop(), no crash

Implementation B C,hatGPT (eligibility_chatgpt.py):

Runs: streamlit run eligibility_chatgpt.py — no import errors, app loads
Core path works: MSTI + "Spr 2026" + Active → click Check Eligibility → 5 rows in st.table()
Invalid input handled: blank quarter → st.warning(); malformed quarter → st.error(); no crash

7. Input Testing Evidence

Input Implementation A (Cursor) Implementation B (ChatGPT)

1 (valid)Program: MSTI, Quarter: Spr 2026, CPT: Active, All 5 rows shown; Employer Panels is Eligible (within 4Q), CPT Sessions is Eligible; green/red styling Same eligibility logic, static st.table(), no color

2 (invalid)Program: Other, Quarter: abc, CPT: Activest.warning() shown, st.stop() prevents table from rendering; no crash st.warning() for "Other", no table shown, no crash

Conclusion: Both implementations handle the required cases. Implementation A is more maintainable typed, documented, functions separated. Implementation B is a closer match to a quick prototype style but still safe for invalid inputs.